

WEB3 TECHNOLOGIES, INC. (DBA ANKR)

Token Optimized RPC

Reading a Blockchain Without Drowning an Agent in Tokens

Mike Kondratev ▪ Alexander Kolesov ▪ Roman Fasakhov ▪ Stanley Wu

Whitepaper draft 0.9, describing TORPC/TOEVM spec v1 ▪ July 2026

*The chain answers in bytes. The agent pays in tokens.
TORPC stops it paying for noise.*

When an AI agent reads a blockchain through a JSON-RPC endpoint, it pays for every byte the node returns: hex strings a model must decode in its head, a `logsBloom` filter it never reads, repeated field names, structural punctuation. Those bytes become input tokens on every call, and for a receipt-heavy workload they dominate the bill and crowd the context window. TORPC is a token-optimized rendering of the same data: the node's response is decoded, renamed, and stripped of what an agent cannot use, negotiated by a single HTTP header. Nothing is invented, and the raw response is one re-query away. It is not a new API and needs no client schema. On a measured production run the semantic tier cuts tokens per response by 48% overall and up to two-thirds on receipts, and a three-model accuracy benchmark shows the model answers *better* from the smaller payload than from raw JSON-RPC. The operator does the decoding once, server-side, so every agent downstream stops paying for it.

Abstract

An AI agent that reads a blockchain through a hosted RPC endpoint pays a tax it never chose. JSON-RPC was designed for programs, not language models: numbers arrive as hex strings the model must convert, addresses in mixed case, event logs as undecoded topic-and-data blobs, a 256-byte `logsBloom` on every receipt, and field names repeated for every element of every array. A program ignores the cost; a model pays for all of it in input tokens, on every call, and a single receipt-heavy read can cost a thousand tokens of which a fraction is signal. TORPC removes that tax without changing the API. A client opts in with one request header; the endpoint returns the same data re-rendered for a model: hex converted to decimal, addresses checksummed, logs ABI-decoded and collapsed, dead fields dropped by published per-method lists, keys normalized. Nothing is invented: the tier-1 transforms on carried fields are mechanically reversible, the enumerated service fields are dropped by policy, and they are recovered by re-issuing the request without the opt-in header against the same block. The transform is tiered (mechanical or fully decoded), deterministic (identical input yields identical output, so prompt caches hit), and conservative by rule: unenumerated fields pass through verbatim, failed transforms fall back to the verbatim field, and anything the server cannot decode is kept raw and flagged rather than dropped or guessed. In a measured run against the live production proxy the covered read methods shrink by 35% at the mechanical tier and 48% at the semantic tier, token-weighted, with receipts at -64.5% and calldata-bearing transactions at -69.0%; on a three-model accuracy suite the semantic tier *raises* retrieval accuracy on every model tested - all three land at 91-93%, against 69-82% from raw - while cutting the suite's corpus by 45% of its tokens. The format is small by design - one header pair plus public per-method mapping rules, EVM first with the pattern reserved for other chains - published CC0-1.0 with Apache-2.0 reference tooling, and deliberately aligned with existing agent-API standards rather than built as a proprietary stack.

Contents

1	The tax you didn't ask for	3
2	The idea	3
3	What stays the same	5
4	How it works	6

5	The correctness contract	9
6	A standard, not a proprietary stack	10
7	Results	12
8	Limitations and what we do not claim	14
9	Related work	16
10	Conclusion	17

1. The tax you didn't ask for

A blockchain node speaks JSON-RPC, a format designed in 2010 for programs talking to programs. A program does not care that a block number arrives as "0x14913f6", that a receipt carries a 256-byte bloom filter it will never inspect, or that a hundred-element log array repeats the same eight field names a hundred times. It parses the bytes, pulls the fields it wants, and discards the rest at no cost.

A language model is not a program, and it pays differently. Every byte in the response becomes an input token the moment the agent puts it in context. The hex it has to convert in its head, the checksummed and non-checksummed addresses it has to reconcile, the `logsBloom` that means nothing to it, the structural braces and repeated keys of pretty-printed JSON: all of it is billed, on every call, at the model's per-token price. And it competes for the context window, so a few receipt reads can crowd out the reasoning the agent was supposed to do.

The numbers are not marginal. Even the receipt of a plain, single-log USDC transfer weighs about 700 tokens, roughly 75 of them the bloom filter alone; a receipt for a busy contract call runs to thousands, most of it structure and undecoded blobs rather than the handful of facts the agent needs: who called what, did it succeed, what did it transfer, what did it cost. Multiply by the call volume of an analytics pipeline, a wallet copilot, or a trading agent, and the token cost of *reading* the chain becomes a real line item, sometimes larger than the model reasoning it feeds.

The pain is not specific to blockchains: the MCP issue tracker carries a proposal literally titled "Mitigating Token Bloat" and standing requests for hard response-size limits [7, 8], and a general-purpose compression layer for agent tool output collected tens of thousands of GitHub stars within months of appearing (as of July 2026) [9]. What is specific to blockchains is that the data is unusually hostile to a model (pervasive hex, undecoded ABI, filters and blooms) and unusually mechanical to fix, because the node already knows the ABI and the types. The decoding a model would do, token by token, in its head, the server can do once, for everyone, before the bytes are ever billed.

2. The idea

The mechanism is small enough to state in one sentence: let a client ask, with one HTTP header, for the node's response re-rendered for a language model, and have the endpoint return the same facts decoded, normalized, and stripped of what a model cannot use, with the standard raw response one re-query away.

Three properties make it safe to turn on by default. First, it is **opt-in and recoverable**: an unmodified client sees standard JSON-RPC; the tier-1 transforms on carried fields are mechanically reversible; and the enumerated service fields the transform drops by policy are recovered by re-issuing the request without the opt-in header, against the same block. That re-query is content negotiation against a live node rather than a retention contract, so archival availability is not guaranteed and a client that needs reproducibility pins a block. Second, it is **tiered and best-effort**: the client names the *highest* tier it will accept, from a mechanical pass (hex to decimal, dead-weight drops) to a full semantic decode (ABI-resolved logs and calldata), and the server serves the highest tier it can honor for that method and that payload - never more. Third, it is **deterministic**: at a given tier, the same raw response renders to the same bytes from a given server (and to the same fields and values from any conforming one - §5), so response-level

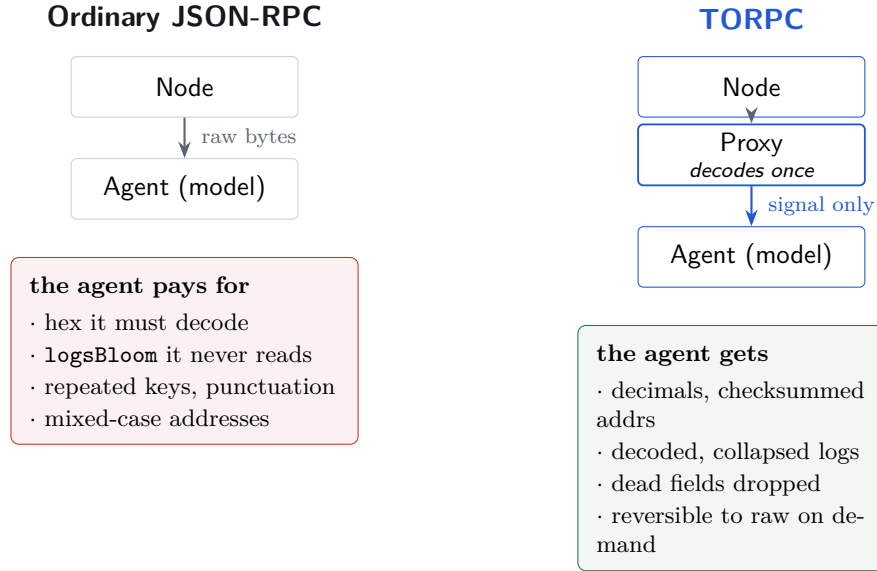


Figure 1. *The work does not disappear, it moves. Ordinary JSON-RPC ships the node's raw bytes and the model pays, in tokens, to decode them on every call. TORPC does the decoding once in the proxy, server-side, so the agent receives only what it can use and every agent downstream stops paying for the same conversion.*

caches (HTTP, client-side memoization) key on stable content, and repeated identical reads stay prefix-cacheable on the model side.

Where this matters most. The token you save scales with how often you read and how noisy the read is. A high-volume agent living on receipts, logs, and traces (analytics, indexing, wallet copilots, trading infrastructure with an LLM in the loop) is the case for it; there, half the tokens is half the bill and twice the headroom in context. For an occasional, small read, plain RPC is fine. TORPC is for the workloads where reading the chain is a cost center, not an afterthought.

Why the server does it - and why it doesn't have to. The mechanical tier is a pure function of the raw response, so nothing stops a client from running the same transform locally; the reference mapper ships as a package in the code repository (@torpc/decoder, structural only: no ABI, no decoding, and source-only for now rather than on a public registry) precisely so it can, and a local transform earns the identical token saving with no trust placed in the provider. What TORPC standardizes is the *rendering*, not the placement: because the per-method rules are published and deterministic, a local transform and a conforming server produce the same bytes, and the header negotiation is simply the deployment that reaches clients which cannot run a transform at all - a bare `curl`, an MCP tool call, a hosted agent runtime with no preprocessing hook. The semantic tier is where the server placement earns its keep: ABI resolution wants a curated, continuously updated signature-and-contract corpus, which is operational data a provider maintains anyway and most clients do not want to host, and running it once on the response path amortizes the decode across every agent downstream instead of re-implementing it in each harness. A client that prefers sovereignty over convenience loses nothing by decoding locally; a client that cannot, opts in with one header.

3. What stays the same

The first question to ask of any transform that rewrites data is what it can lose. For TORPC the answer is the point of the design, so it goes up front. Here is exactly what a TORPC response preserves, and what it deliberately changes.

The API does not change. TORPC is not a new protocol with new methods. It is a rendering of the responses of the JSON-RPC methods that already exist. A client calls `eth_getTransactionReceipt` the way it always did; the only difference is one request header and the shape of the value that comes back. The JSON-RPC 2.0 envelope is preserved verbatim (`jsonrpc`, `id`, the `result/error` frame), and JSON-RPC *error* responses are never touched at any tier - an `error` object comes back exactly as the node produced it, so failure handling looks the way it always did. There is no schema to ship, no SDK required, no new endpoint to learn. A client that does nothing gets ordinary JSON-RPC. The corollary deserves its own sentence: the rendered `result` is meant for a model's context, not for typed client libraries - the renames and re-typed values that help a model will break a parser expecting standard field names, so do not enable the opt-in on a transport shared with viem/ethers-style code. Opt in on the path that feeds the model; leave every other connection untouched.

Nothing is dropped silently, and nothing is guessed. Every field of the raw response falls into one of three published classes. It is *carried* - usually re-typed into a form a model reads directly (hex integer to decimal string, address to its EIP-55 form) or renamed to one canonical key. Or it is *redundant within the response* and collapsed: a receipt's logs each repeat the receipt's own block number, block hash, and transaction hash, and TORPC keeps those once at the top level instead of once per log. Or it is *node-internal bookkeeping* - the 256-byte `logsBloom` (derivable from the very logs it summarizes), running counters like `cumulativeGasUsed`, envelope tags - dropped by an enumerated, published per-method list, never ad hoc. Two hard rules police the boundary. Fields *not* named in a method's ruleset - a chain-specific extension such as Optimism's `l1Fee*` or Arbitrum's `gasUsedForL1`, or a field added by a post-spec EIP - **must** pass through verbatim, untransformed. And a transform that fails on a malformed or unexpected field **must** fall back to the verbatim field rather than drop it, invent a value, or abandon the rest of the response. At the semantic tier, an element the server cannot decode - an event whose ABI it cannot resolve - is kept raw and flagged (`_event_unknown`, `_function_unknown`); the spec's normative language is blunt: servers **must not** invent a decoding. A decoded receipt with fourteen logs where one is unknown returns thirteen decoded and one raw, never thirteen.

The raw response is one re-query away. The escape hatch is HTTP content negotiation, not a retention contract: re-issue the same request, with the same authentication, without the TORPC opt-in, and the endpoint returns the standard JSON-RPC response. Two limits come with that. For snapshot consistency across the two calls a client pins a block number or hash in the underlying call, exactly as it would with any RPC provider - behind a load-balanced endpoint consecutive answers may come from different nodes, and a reorg between the two reads is a property of Ethereum, not of TORPC. And the re-query can only return what the node still serves: archival depth is an operational property of the deployment, not a retention promise attached to the format, so an agent that needs a rendering to stay reproducible pins the block it read.

What changes, and what it costs a reader

- **Numerics become decimal strings.** "0x17ee5d9" → "25093593". A model reads it directly; no in-head hex conversion, and exactness survives any magnitude because the value stays a string. A few enumerated status codes additionally render as words ("0x1" → "success") - a value map published in the ruleset like any other rule.
- **Addresses become EIP-55 checksummed strings** [1]. One canonical form, so the model does not reconcile cases.
- **Keys are renamed once, per a published per-method map** (transactionHash → tx, effectiveGasPrice → gas_price), and enumerated dead-field lists drop node bookkeeping (logsBloom, cumulativeGasUsed). Never ad hoc: unenumerated fields pass through verbatim.
- **Logs and inputs are ABI-decoded** into named events and functions with named arguments. *Caveat:* argument names are derived by the proxy and may differ from the canonical ABI (an ERC-20 transfer may decode as recipient/amount rather than to/value); do not assume canonical names.

4. How it works

Four moving parts: the tier ceiling a client names, the header pair that negotiates it, the server-side transform, and the escape hatch back to raw. Everything happens inside **result** on success responses; the envelope and all error responses are out of bounds by rule.

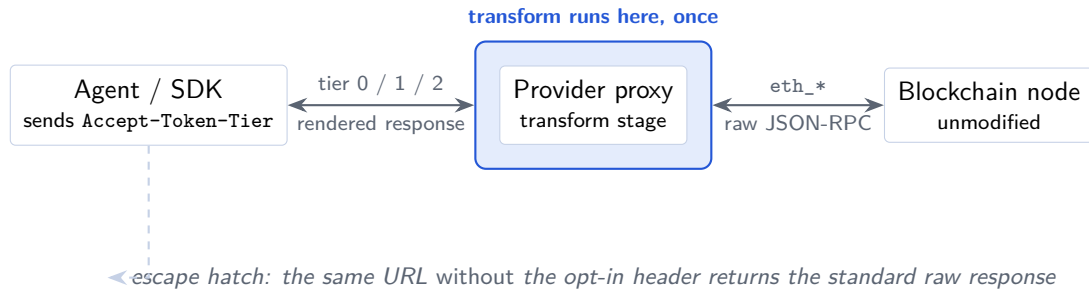


Figure 2. A client negotiates a tier with one request header; the transform (in Ankr's deployment, a stage of its production proxy) decodes the node's raw JSON-RPC response server-side and returns the rendered form. The node is unmodified. Dropping the header on the same URL returns the raw response, so the transform is never a one-way door.

The tiers. A client asks for exactly as much rendering as it wants.

The header pair. Negotiation is one request header and one response header, and nothing else. **Accept-Token-Tier:** N (with N in $0..2$; higher values are reserved) is a *ceiling*, not a demand: the client states the highest tier it will accept, and the server may serve any $M \leq N$ - best-effort by design, because a method may have no rules at the requested tier or the ABI may be unavailable for this payload. The ceiling semantics also make out-of-range values forward-compatible: a v1 server answering a reserved tier simply serves its highest supported one. The response carries **Token-Tier:** M , the tier actually applied, on *every* response to an opted-in request - including $M = 0$, which says "spec supported, nothing transformed here," and including

Tier	Name	What the server does
0	raw	Nothing. Standard JSON-RPC 2.0, untouched - the default, and the mandatory answer for any method without a published ruleset.
1	mechanical	Field-level transforms with no external lookup, a pure function of the raw response, mechanically reversible on every field they carry: hex numerics to decimal strings, EIP-55 casing on address-typed fields, canonical renames, the enumerated dead-field drops, per-log redundancy collapse, zero-padding strip on 32-byte values.
2	semantic	Tier 1 plus ABI-aware decoding of event logs and transaction calldata into a fixed shape: <code>{"event"/"function", "args"}</code> with named arguments. Per-element and best-effort; an undecodable element is kept raw and flagged. Faithful to the resolved decoding where one is available, never invented where it is not.
≥ 3	-	Reserved. A v1 server must not emit a tier above 2; richer renderings stay unspecified until they have a normative design and a determinism contract.

JSON-RPC *error* responses, which pass verbatim and carry **Token-Tier: 0** (verified against the production endpoint; a transport-level failure such as an HTTP 401 is not a JSON-RPC response and carries no tier header). A response with no **Token-Tier** header at all means the server does not implement TORPC, and the client treats the body as ordinary JSON-RPC. That asymmetry is the whole deployment story: a server that has never heard of the header simply ignores it, so an agent can send the opt-in everywhere and degrade cleanly. (One failure mode to name: a middlebox that strips unknown response headers defeats this detection; decoded elements are self-identifying by shape - the `event/function` keys - so a client behind an aggressive proxy can still verify what it received.) Whether to surface an ABI-version signal for cache coordination is an open question the spec tracks explicitly.

Batches and caches. A JSON-RPC batch is transformed element by element, each at $\min(\text{requested ceiling, the method's maximum tier})$, and coverage differs across the elements: an uncovered method inside a batch passes through verbatim, and so does an element carrying a JSON-RPC error (applied tier 0). One header therefore has to describe a mixed response, and v1 resolves that by making the header a floor rather than a per-element contract. **Token-Tier** on a batched response carries the *minimum* tier applied across the batch, elements may sit above it, and a client parsing a batch determines each element's tier from its shape rather than from the header. The transforms are self-identifying by construction, since tier 1 renames fields and emits decimal strings where the raw form had `0x` hex, and tier 2 replaces raw log fields with `event` and `args`. The alternative, clamping every element down to the minimum, was rejected because it removes the saving exactly where batching is most common: a batch of a scalar method next to `eth_getLogs` would drag the logs back a tier because the scalar has no tier-2 ruleset. Per-element shape handling is not a new obligation either, since `_event_unknown` already means a tier-2 response carries tier-1 log entries inside it.

Caches need the same discipline. Because the body of a response now depends on request headers, a conforming server **must** emit an HTTP **Vary** header naming every request header that influences the transform (**Accept-Token-Tier**, plus any deprecated alias the deployment still honors), and **must** emit it on *every* response of the endpoint rather than only on opted-in ones. The unconditional form is the point: the poisoning case is a non-opted raw response stored

by a shared cache without **Vary** and then handed to a client that asked for a rendered one, or the reverse, and a server that emits **Vary** only when it transformed something leaves exactly that hole open. Both rules are normative requirements of the spec rather than descriptions of any particular deployment, and they are newer than the transform itself, so a client that sits behind a shared HTTP cache should verify the header is present instead of assuming it.

The transform. The rendering runs client-side on the response path - in Ankr's deployment, a stage of its production proxy - dispatched per method against the published rulesets, over JSON-RPC 2.0 on HTTP (WebSocket subscriptions are out of scope for v1). Coverage today, verified by a live probe of the production endpoint (2026-07-17): ten methods reach full tier 2 - the block-and-transaction read family (receipts, logs, block receipts, blocks by hash or number, transactions by hash or by block-and-index) plus the two uncle-by-index methods; twelve scalar methods whose entire result is a single hex integer (`eth_blockNumber`, `eth_getBalance`, `eth_chainId`, ... - collectively among the highest-volume shapes on the tier) plus `eth_feeHistory` take the mechanical tier - twenty-three transformable methods in all; every other method returns raw with **Token-Tier: 0** rather than erroring, so a client always gets a valid response - it just may not get a saving. Best-effort has one more face worth naming: the transform applies up to an operator-configured response-size threshold (about 5MB in Ankr's deployment at the time of writing); a response above it skips the transform and passes through raw with **Token-Tier: 0**, by the same never-error rule. The full method-by-method coverage table, with the tier each method reaches and a link to its ruleset, lives in the spec repository at [specs/methods/README.md](#). Two deliberate exclusions are worth naming. First, `eth_call` return values are *never* decoded at any tier: the caller built the calldata, so the caller owns the matching ABI, and a server-side decode adds nothing while risking divergence from it. Second, the response never echoes request-derived values (target address, selector, block tag) back at the client: the requester still holds its own request, and on small reads the measured cost of such an echo, 25-40 tokens, often exceeds the payload itself.

The transform described here is live on Ankr’s EVM endpoints (rpc.ankr.com) across the EVM chains it serves, on both the public and keyed tiers, included at no additional charge; the spec repository’s README carries a quickstart that runs against a keyless public endpoint and links the per-method coverage table.

The spec's worked receipt (a USDC transfer, block 25,093,593): raw (grey) → tier 2 (blue)

```
"blockNumber": "0x17ee5d9"
  → "block": "25093593"

"status": "0x1"
  → "status": "success"

"effectiveGasPrice": "0x8b485351"
  → "gas_price": "2336772945"

"logsBloom": "0x000...(256 bytes)"
  → (dropped - derivable from the logs below)

per-log "blockNumber" / "blockHash" / "transactionHash" / "logIndex" / ...
  → (dropped - duplicates of the receipt's own fields, kept once)

"logs": [{"topics": ["0xddf252ad...", ...], "data": "0x...06fc23ac00"}]
  → "logs": [{"event": "Transfer", "contract": "0xA0b8...eB48",
    "args": {"from": "0x2744...a22b", "to": "0xfa21...e7fa", "value": "30000000000"}}]
```

On this receipt the spec's own worked numbers are 696 tokens raw, 342 at tier 1, and 292 at

tier 2 (−50.9% and −58.0%); token counts here and throughout are `o200k_base` (OpenAI’s tokenizer, used for model-independent measurement). The example, the tier echoes, and the tier-0 passthroughs above were re-measured against the production endpoint on 2026-07-27.

Discoverability. Nothing about TORPC is machine-discoverable today, and v1 does not pretend otherwise: an agent learns the format from the published spec, or from a client library that already knows it, or by sending the header speculatively and reading whether a `Token-Tier` comes back. Two routes are planned rather than shipped. An MCP server built on TORPC (`@asphere/agent-rpc-mcp`) is to carry a `.well-known/torpc` manifest and an `llms.txt` in its own distribution, but it is not published to any package registry yet, so no tool catalogue lists it. Endpoint-level discovery - an RPC endpoint advertising its tiers and method coverage at a well-known path, per the IETF APIx-core draft [5] - is the recorded design direction, but it is not normative in v1 and not served on the RPC endpoints themselves. v1 deliberately keeps its normative surface to the header pair and the per-method rules.

5. The correctness contract

Here is exactly what a client gets, stated once and precisely.

The correctness contract

A TORPC response accounts for every field of the raw response in one of three published ways: carried (re-typed into a form a model reads directly), collapsed as redundancy within the same response, or removed by an enumerated per-method list of node-internal bookkeeping - never silently, never ad hoc. Fields outside a method’s ruleset pass through verbatim; a transform that fails falls back to the verbatim field; a decode the server cannot resolve is flagged per element, never invented. The standard raw response stays one opt-out request away; at a given tier and decoding, identical input renders to identical bytes from the same implementation, and to the same fields and values from any conforming one.

The contract is about the rendering, not the answer: it says a TORPC response faithfully represents what the node returned, not that the node’s answer is correct (§8). The claims behind it, each stated with its bound:

C1 - every field is accounted for. Each field of the raw response is either carried (possibly re-typed: hex to decimal string, address to EIP-55, one canonical rename), or collapsed as in-response redundancy (per-log copies of the receipt’s own block number, block hash, and transaction hash are kept once at the top level - still present, just not repeated), or dropped by the method’s enumerated dead-field list. The dropped class is bookkeeping the response’s primary reader - a model answering questions - does not consume: `logsBloom` is derivable from the very logs the response carries; `cumulativeGasUsed` and the EIP-2718 `type` tag do have consumers (indexers, gas accounting), but those consumers are tier-0 readers by definition, and the fields stay one opt-out request away. Every removal is a published, reviewable line in the method’s mapping document - not a runtime judgment call - and a drop that proves wrong for real workloads is a one-line ruleset revision, not a format change. The boundary is policed by two normative rules: unenumerated fields pass through verbatim (so chain-specific extensions and future EIP fields survive untouched), and a transform that fails on a field must preserve

that field verbatim rather than drop or repair it. This is a structural property of the per-method rulesets, pinned by their worked examples, with the machine-checkable golden vectors still at scaffold stage (§6), not a statistical claim.

C2 - decoding is faithful or absent, never invented. A decoded element is decoded against the resolved signature and ABI; an element the server cannot resolve is passed through raw with an `_event_unknown` / `_function_unknown` flag, and the spec forbids emitting a decoding the server is not sure of. The honest seam is what “resolved” rests on: for event logs the resolution is anchored by the full 32-byte topic hash, for calldata by a 4-byte selector, and the two are not equally strong - §8 takes this up as a security consideration. Argument *names* additionally come from the server’s ABI source and may differ from the canonical ABI (`recipient/amount` for an ERC-20 `transfer` rather than `to/value`); the decoded *values* are the call data itself.

C3 - the raw response is always one request away. Because the escape hatch is content negotiation against the same endpoint, not a stored copy, recovery does not depend on any retention window or extra infrastructure. The caveats are the ordinary ones of any load-balanced RPC service: consecutive requests may be answered by different nodes, and a reorg between the two reads can change the underlying data - both are avoided the standard way, by pinning a block number or hash in the call.

C4 - the rendering is deterministic, with one scoped seam. Determinism is claimed at two deliberately different strengths. *Within* an implementation, the rendering is a pure function: the same raw response at the same tier renders to the same bytes, which is what makes TORPC safe for caching - a client’s response cache, an HTTP cache, and prefix caching all key on stable content. *Across* implementations, the spec pins the field *set* and the transformed *values* (structural equivalence, checked by the golden vectors), not the byte layout: JSON serializers do not guarantee key order (Go’s, notably, does not), and mandating a byte canon across independent servers would buy little - outside cryptographic contexts nobody pins serialization order, and no client keys a cache on another server’s bytes. The semantic tier adds the one real seam: decoding is provider-defined, so a cross-server (or cross-month) rendering diverges where ABI coverage differs - a server that learns a new ABI starts decoding an element it previously flagged. The failure mode is benign by construction: a changed rendering is a cache *miss*, never a wrong answer, and whether servers should surface an ABI-version signal for cache coordination is an open question the spec tracks.

6. A standard, not a proprietary stack

The fastest way to make a wire format irrelevant is to ship it as a vendor protocol nobody else can adopt. TORPC is built the opposite way, and this section is about the seams a careful reviewer pushes on: is this a real standard, or Ankr’s private encoding with a public name?

The spec is small, layered, and open. The normative surface is deliberately tiny: one core document (the EVM compression spec - tiers, the header pair, the passthrough and determinism rules) plus one mapping document per covered response shape, each carrying the exact field-level rules, edge cases, and a worked example, with a coverage table that resolves every method to its document (one document covers the twelve hex-integer scalars; the by-hash and by-number forms of a method share one). TORPC, Token Optimized RPC, is the umbrella name for the family; the EVM binding (TOEVM - the name behind the `@torpc/toevm-rules` package) comes first, and the same pattern is reserved for other chains (TOSOL, TOSUI, TOTRON). The spec is published

CC0-1.0, submittable as an EIP without a licensing obstacle, and the reference tooling is Apache-2.0. CC0 waives copyright but grants no patents, so the patent position rides on the reference implementation’s Apache-2.0 licence and its express patent grant; and because CC0 does not touch trademark, a trademark policy and a “TORPC Conformant” certification claim keep the name attached to behavior a conforming implementation can be checked against. The format is free to copy; the claim that a copy is conformant is not. That claim is also not open yet: with the vector suite at scaffold stage (§6), no implementation may assert conformance today, Ankr’s own included. An earlier, envelope-based draft of the format is retired but retained in the repository for the record: the v1 design deliberately moved everything out of the body and into headers after engineering review.

It aligns with existing agent-API standards rather than inventing parallel ones. This is deliberate, and it is where a proprietary format would show. The v1 spec itself claims nothing beyond its two headers and the per-method rules; for every adjacent surface, the project’s recorded design decisions adopt a standard already in flight rather than author a private equivalent. Token budgeting tracks the IETF work on bounded agent authority: the decision was recorded against `draft-mcgraw-httpapi-agent-budget-01` (HTTP 427 plus a `Budget-Attestation` credential [2]), and that draft has since been restructured as a delegation authentication scheme carrying a COSE/CBOR budget attestation over ordinary 401/403 challenges. What TORPC commits to is the posture, not the revision: blockchain-specific extension keys contributed to whatever the work lands on, never a private `X-Token-Budget`. Field projection tracks JSON:API sparse fieldsets [3] (`?fields[receipt]=tx,block,fee`) rather than a bespoke array. Agent identity tracks the OpenAPI `x-agent-trust` extension [4]; discovery tracks APIx-core [5] (§4). Each choice trades a cosmetic difference for the right to read as “an extension of an established standard” instead of “yet another vendor stack” - and none of them is allowed to grow the core spec.

The rendering is auditable, and the audit trail is the spec itself. Every per-method document carries the complete field-level mapping table and a worked example with real payloads and real token counts, so a reviewer can check any claim in this paper against the rules and a third party can implement the transform without talking to us. Around the documents sit the machine-checkable pieces, at an honestly early stage: a typed ruleset package (`@torpc/toevm-rules`, Apache-2.0) and a reference mapper for the mechanical tier (`@torpc/decoder`), both source-only in the code repository rather than on a package registry, and a golden-vector conformance harness that is deliberately labelled a scaffold, v0.0.1: a dependency-free runner that exercises an implementation only through an adapter, and a single case (a receipt’s hex-strip and service-field drop), growing method by method. That case pins the tier-1 and tier-2 expectations for one production-captured receipt, so a single receipt is the entire machine-checked surface today. Nothing in that set should be read as coverage today. The direction is that §5 becomes machine-checkable end to end - today the per-method mappings are the normative anchor and the vectors are catching up to them.

Where TORPC sits next to TOON. A general token-oriented encoding like TOON [6] compresses *any* JSON by removing structural redundancy, and it composes with TORPC: TOON is a lower encoding layer, TORPC is the blockchain-semantic layer above it. TORPC’s work (knowing that `0x1` in a receipt’s `status` means success, that a topic array is a `Transfer`, that `logsBloom` is dead weight for a reader) is exactly the domain knowledge a general encoder cannot have. The two are complementary, not competing.

7. Results

Two things have to be true for the idea to pay off: the rendering has to remove a lot of tokens, and it has to not cost accuracy. Both are measured. The accuracy suite and the pinned token corpus are reproducible from the public benchmark; the live production run is reproducible from the measurement script published beside it.

Token reduction (live production run). The published measurement against Ankr’s production proxy, 2026-07-17, covers 25 random mainnet blocks and 525 raw/tier-1/tier-2 triples over the 21 methods that return data on post-merge mainnet (of 23 transformable: the two uncle methods answer `null` and produce no measurable triple), counting o200k over the full HTTP response body. Tier 1 removes 35.3% of tokens and tier 2 48.4%, token-weighted. The measurement script and its per-method table are published in the benchmark folder of the code repository, `bench/scripts/live-token-savings.mjs` and `bench/results/live-token-savings-2026-07-17.md` in `github.com/w3tech/torpc-js`. A prior six-method run seven weeks earlier measured -34.0% / -45.6% , and the public benchmark corpus lands in the same place independently (-45.0% at tier 2 over its 114 pinned fixtures).

Two labels matter whenever any of this is quoted. The token-weighted overall is dominated by the giant payloads; the unweighted per-method mean over the same run is -17.4% at tier 1 and -24.5% at tier 2, dragged down by the twelve scalar methods whose entire response is about twenty tokens. Both describe the same data, and neither belongs in a sentence without its label. The spread is the whole story anyway, because the saving tracks how much of a payload is decodable noise. Per method at tier 2 the live run spans -69.0% on `eth_getTransactionByHash` (where the calldata decode lands) and -64.5% on receipts, through -40.4% on a full block and -28.9% on `eth_getLogs`, down to the hex scalars where a handful of tokens is all there is to remove. The pinned public corpus shows the same shape sample by sample: a Uniswap V3 swap receipt at -62.0% and an NFT-mint receipt at -63.0% . The receipt worked end to end in the spec’s own method document, a single-log USDC transfer measured outside both runs, goes from 696 tokens raw to 292 at tier 2 (-58.0%).

Latency comes down with the tokens. A server-side decode on the hot path invites the question of what it costs in milliseconds; measured end-to-end, the answer is that it pays for itself. On cold connections (fresh TLS per request, twelve-sample runs from a single client location) the smaller transfer wins outright: a receipt read moved from 300 ms p50 raw to 220 ms at tier 2, and an `eth_getBlockReceipts` call from 1,018 ms to 618 ms (821 KB to 316 KB on the wire). On warm connections (the 25-sample live run, per-method p50) tier 2 sits at parity or faster on every method except the single heaviest, where the transform’s cost finally shows: `eth_getBlockReceipts` at +16 ms on a payload whose raw form is ~ 1.5 MB. One vantage point and modest sample sizes - but the direction is consistent: the rendered response arrives sooner, or within milliseconds, never meaningfully later.

Accuracy (does the smaller payload still answer the question?). The concern with any lossy-looking rendering is that the model gets worse. Measured, the opposite happens. `agent-rpc-bench` pins real mainnet responses at one block across the covered methods (plus three uncovered ones as a control) and asks 100 questions over each of the three formats - 300 evaluations per model, answered through the plain chat-completion API with *no tools*, *value-only*, and scored deterministically against typed ground truth, no LLM judge. That protocol is the point: it measures *format comprehension* - what a model gets right from the bytes alone -

Method (sample)	raw tokens	tier-2 tokens	reduction
eth_getBlockReceipts	384,235	152,562	−60.3%
eth_getBlockByNumber (full transactions)	231,032	137,781	−40.4%
eth_getLogs (3 blocks, one ERC-20 Transfer topic)	48,342	34,378	−28.9%
eth_getTransactionReceipt	4,570	1,624	−64.5%
eth_getTransactionByHash	1,819	564	−69.0%
eth_feeHistory	242	188	−22.2%
twelve hex-integer scalars (each)	18-27	16-22	−9 to −19%

Table 1. Per-method token reduction, tier 2 vs raw for identical calls: per-method means over the 25 random mainnet blocks of the live production run (o200k tokenizer, 2026-07-17; the same run behind the −48.4% aggregate, so the table and the headline share one source). The raw baseline was separately cross-checked against an independent provider’s output for the same calls: our tier-0 came in 0.52% larger in aggregate, with four of seven responses byte-identical and the entire difference explained by one field, `blockTimestamp`, which our nodes emit on transaction objects and the other endpoint does not. Since tier 1 drops that field, quoting a saving against our own raw is very slightly generous on transaction-shaped payloads, by about two percent there and under one percent in aggregate. The run, the per-call table and the script are published with the benchmark harness. The token-weighted aggregate is dominated by the two heaviest rows; the honest reading is per row: savings track decodable structure, so receipt-shaped and calldata-bearing payloads compress hardest while a twenty-token scalar has almost nothing to give.

not an agent harness’s ability to run code over the payload.

Model	raw	tier 1	tier 2	Δ raw $\rightarrow$ tier 2
Claude Haiku 4.5	69.4% (59/85)	85.9% (73/85)	90.6% (77/85)	+21.2 pp
Claude Sonnet 4.6	82.4% (70/85)	90.6% (77/85)	92.9% (79/85)	+10.5 pp
Claude Opus 4.8	77.6% (66/85)	87.1% (74/85)	91.8% (78/85)	+14.2 pp

Table 2. Retrieval accuracy by tier, per model, on the shared 85-question bucket: the items whose payloads fit the smallest tested window (Haiku’s 200K) at every tier, scored identically for all three models so the comparison stays apples-to-apples. Sonnet and Opus ran with a 1M-token window and scored all 100 questions; their full-suite results appear in the capacity discussion below. One sample per item, so read each model’s own raw $\rightarrow$ tier-2 delta - the suite is not a cross-model ranking.

Tier 2 lands every model, weak or strong, at 91-93%: the lift is the format, not the model, and it is largest at the cheap end of the ladder. The question mix matters for reading that lift honestly, so here it is by category (tier-2 accuracy, Haiku/Sonnet/Opus): retrieval 87/97/97%, ABI-event decoding 87/100/100%, validity 100/100/100%, filtering 50/83/67%, aggregation 56/67/67%, structure 14/71/71%. The lift concentrates exactly where the format does work the model otherwise does in its head - decoding and numeric retrieval - and the three uncovered methods, which pass through identically at every tier, serve as a built-in no-op control. The mechanism shows up in single items: asked for `eth_blobBaseFee` from the raw response, a model converts the hex in its head and returns 348891897 (a mis-conversion); handed the tier-1 decimal, it returns 21869561, correct - same data, and the format removes the error class. The suite also shows what the format cannot fix: aggregation questions over thousand-entry payloads (count the distinct addresses across 486K tokens of logs) are not reliably answerable by *any* model without running code, and they drag that category for raw and tier 2 alike - a protocol limit,

not a format signal.

Capacity is a second axis, kept apart from accuracy. Fifteen of the hundred questions sit on giant payloads - full blocks, wide log scans, block receipts at 320K-580K tokens raw. On the 200K-window model (Haiku) all fifteen overflow at every tier, measured empirically: the API rejects the request. (Token figures here are o200k counts, which run below Anthropic’s own tokenization on this material - the overflow itself measures the gap: a payload at a nominal 180K o200k exceeding a 200K billed window puts the density factor at roughly 10% or more on these items.) Overflow is scored as overflow, never as a wrong answer: folding it into accuracy would conflate “answered wrong” with “could not read it at all,” two different product problems with different fixes. The two 1M-window runs (Sonnet, Opus) ingested all fifteen and answered them at 60-67% from raw versus 86.7% at tier 2 - the giant payloads are exactly where the decoded rendering helps most, and compression cuts what everything beneath the capacity line costs.

By the numbers (agent-rpc-bench, API path, three Claude models)

- **Accuracy (shared 85-question bucket):** Haiku 4.5 69.4% → 90.6%, Sonnet 4.6 82.4% → 92.9%, Opus 4.8 77.6% → 91.8%, raw → tier 2 - and the climb is monotone through tier 1 on every model.
- **Suite tokens:** 1,871,995 → 1,029,566 o200k across the covered methods (−45.0%); receipts −54 to −63%, and the suite’s single largest reduction is a calldata-heavy transaction at −96.5% (16,636 → 574: the decoded `function+args` replaces a giant `input` blob). o200k counts understate Anthropic-billed savings (see the capacity note above).
- **Capacity, kept separate:** 15 of 100 questions overflow the 200K-window model at every tier (320K-580K tokens raw, verified empirically) - scored as overflow, never as misses; the 1M-window runs ingest all fifteen and tier 2 answers 86.7% of them vs 60-67% from raw.
- **Live production run:** −35.3% (tier 1) / −48.4% (tier 2), token-weighted over the 21 methods that carry the measurement (−17.4%/−24.5% unweighted per method); 525 triples over 25 random mainnet blocks, script and per-method table published in the code repository’s `bench/` folder.
- **Honest gap:** all three models are the Claude family; GPT and Gemini runs are pending and the claim is scoped accordingly.

8. Limitations and what we do not claim

These are the boundaries of the claims in this paper, stated as precisely as the claims themselves.

- **Rendering, not verification.** TORPC changes how a response reads, not whether it is true. It faithfully re-presents what the node returned; it does not attest that the answer is correct, and a decoded name is a strong hint, not a proof (see the selector note below). Two consequences deserve stating plainly. The raw re-query is an *audit* path, not a hot path: verifying every read doubles requests and re-imports the token cost the format removes, so in practice an agent verifies selectively - before acting on high-stakes state, not on every fetch. And under the semantic tier, byte-level cross-provider comparison stops being a tamper detector: two conforming servers may legitimately render the same response differently (C4), so a rendered response is trusted exactly the way any hosted RPC response is - by trusting

the operator. Response integrity and attestation are a separate concern, out of scope for a token-optimized format.

- **Savings are payload-dependent.** The reduction tracks how much of a response is decodable noise. On the live run receipts average -64.5% and calldata-bearing transactions -69.0% ; across the pinned public corpus individual receipts range from -54% to -63% , the largest single reduction in the suite is a calldata-heavy transaction at -96.5% , and a transaction whose calldata the server cannot decode barely moves at all. `eth_getLogs` lands lower (-27 to -29% across both sets), and a twenty-token scalar or an already-opaque array of hashes gives up almost nothing. TORPC helps in proportion to how much structure there is to remove; it is not a flat percentage, and any savings estimate should be built on a representative workload mix rather than a single best-case method.
- **Semantic enrichment is roadmap, not production.** The shipped transform decodes and normalizes what is already in the response. Higher-level fields that some earlier drafts described (USD prices, an `action.type` classification, known-contract labels, finality and congestion levels) are *not* in production and are not part of the contract in §5. The spec reserves tiers ≥ 3 for exactly this class of rendering and declines to specify them until each has a normative design and a determinism contract; nothing in that class should be presented as live.
- **Decoded names are only as strong as their resolution anchor, and the anchors differ.** An event decode is anchored by the full 32-byte `topics[0]` hash: a signature that matches the hash cannot be spoofed, so the event name and the argument *types* (and hence values) are pinned; only argument *names* are best-effort ABI metadata and may differ from the canonical ABI (`recipient/amount` vs. `to/value`). A calldata decode hangs on a 4-byte selector: collisions across the 2^{32} selector space are cheap to mine, and public signature registries [12] are adversarially writable, so a plausible-but-wrong signature can be registered against a target selector - a known phishing vector in wallet UIs. And one honesty note that cuts the other way: even a perfectly hash-anchored event decode is faithful to the *emitting contract*, not to any brand - any contract may emit `Transfer(address,address,uint256)`, and a scam token's `Transfer` renders identically to USDC's. A decoded name identifies the event's shape; the `contract` address next to it is what identifies the asset. The server's ABI corpus is provider-curated (verified-source registries and signature databases), which is also why argument *names* are best-effort. The spec's mitigations are that a server must never invent a decoding and must flag what it cannot resolve; the operational rule for consumers is that a decoded `function` name is a strong hint, not an authenticated claim - an agent must not make trust or authorization decisions from a decoded name alone, and the raw re-query is the verification path.
- **Decoding enlarges the prompt-injection surface, and TORPC does not sanitize it.** Chain data is attacker-authored: token names, string arguments, revert strings. In a raw response much of it hides inside hex blobs; tier 2 decodes those blobs into text the model reads directly, which is the entire point - and also a larger injection surface. TORPC performs no content filtering and cannot: the strings are the data. Harnesses must treat TORPC output like any other tool output, as untrusted data rather than instructions, and the flagged-raw fallback means even unresolved (opaque) payloads eventually reach the model through other

tools; the defense belongs in the agent harness, not the wire format.

- **A header-dependent body puts two requirements on deployments, and both are newer than the transform.** Once the response body depends on a request header, two failure modes open that convention alone does not close, so v1 makes both normative. On a batched response **Token-Tier** is the minimum tier applied across the batch *and* no element may be transformed above it: without the second half a server can declare the minimum while rendering one element higher, and a client that trusts the header mis-parses exactly that element. On every response of the endpoint, opted-in or not, the server must emit **Vary** naming each request header that influences the transform, or a shared cache can store one variant and hand it to a client that asked for the other. These are requirements the spec places on conforming servers, not claims about what a given deployment does today; both postdate the shipped transform, so a client behind a shared HTTP cache should verify that **Vary** is present rather than assume it. On Ankr’s endpoints the unconditional form is live: **Vary: Accept-Token-Tier, Rpc-Compress, Rpc-Decode** came back on opted-in and non-opted responses alike when this was re-checked on 2026-07-27.
- **Coverage is EVM reads over HTTP.** The block-and-transaction read family reaches full tier-2 and the twelve hex-integer scalar methods plus **eth_feeHistory** reach tier-1 today; uncovered methods pass through at tier 0. Transport is JSON-RPC 2.0 over HTTP - WebSocket subscriptions and other transports are out of scope for v1. Non-EVM chains (the **TO<CHAIN>** bindings) are roadmap. Write methods (**sendTransaction**) are out of scope; TORPC is a read-path format.
- **It composes with context-management layers, and does not replace them.** Techniques that keep raw responses out of the model’s context entirely - code-execution agents, tool-search - attack the same token cost from a different angle [10, 11]. Where an agent already runs full code-mode, TORPC’s marginal token benefit at the context boundary is smaller (though bandwidth and the cost of the occasional response that does reach context still favor it). The layers stack; TORPC is the wire, not the harness.
- **Not claimed:** a new query language, a client-side schema requirement, a replacement for JSON-RPC, or byte-level invertibility of the rendered body (recovery of the raw form is by re-query, not by inverse transform).

9. Related work

General token-oriented encodings (TOON [6]) cut structural redundancy from any JSON and sit one layer below TORPC, which adds the blockchain semantics a general encoder cannot know (§6). The broader agent ecosystem is converging on the same pain from other directions: the MCP tracker’s token-bloat and response-size proposals [7, 8] treat the symptom generically at the protocol layer; Headroom [9] compresses arbitrary tool output between the tool and the model, domain-blind by design; and code-execution agents and tool search [10, 11] keep large responses out of the model’s context altogether. These are context-management layers that compose with a smaller wire, not substitutes for one. Among RPC providers, the adjacent moves are hosted MCP wrappers and x402 pay-per-call rails rather than response compression: context/indexing layers add semantics without shrinking the wire, and the nearest token-aware move - routed domain tools with paging - operates at the tool layer, not the response bytes. As of this draft

no other provider ships a response-format tier negotiation. Historically, schema-driven formats (Protobuf/gRPC, GraphQL) won efficiency by making the client hold a schema; TORPC's bet is the opposite, that a drop-in, opt-in, schema-free rendering negotiated by one header is what an agent ecosystem with no shared schema will actually adopt.

10. Conclusion

An agent reading a blockchain pays, in tokens, for a format built for programs: hex it must decode, filters it never reads, structure it does not need. TORPC re-renders that response for the reader it actually has - decimal, decoded, checksummed, stripped of enumerated dead weight - server-side and once, negotiated by a single request header and always reversible to the raw response. On a measured production run it cuts tokens by a third at the mechanical tier and 48% at the semantic tier, and on a three-model accuracy benchmark the smaller payload answers better on every model tested. The format is tiered, deterministic, conservative by rule - what it cannot decode it flags, what it does not enumerate it passes through verbatim - published open, and aligned with standards already in flight rather than built as a private stack. It does not ask an agent developer to adopt a protocol. It asks them to add one header, and stops charging them for noise.

References

- [1] V. Buterin, A. Van de Sande. *EIP-55: Mixed-case checksum address encoding*. <https://eips.ethereum.org/EIPS/eip-55>
- [2] A. McGraw. *The Budget HTTP Authentication Scheme and 427 Budget Required Status Code (draft-mcgraw-httpapi-agent-budget-01)*, IETF, 10 June 2026. <https://datatracker.ietf.org/doc/html/draft-mcgraw-httpapi-agent-budget-01>
The work was restructured in later revisions as *The Delegation HTTP Authentication Scheme for Request-Bound Authority* (-02, 15 June 2026; -03, 23 July 2026), which carries a COSE/CBOR budget attestation over 401/403 challenges and drops the dedicated 427 status code: <https://datatracker.ietf.org/doc/draft-mcgraw-httpapi-agent-budget/>
- [3] *JSON:API v1.1 - Sparse Fieldsets*. <https://jsonapi.org/format/#fetching-sparse-fieldsets>
- [4] *OpenAPI x-agent-trust specification*. <https://github.com/x-agent-trust/spec>
- [5] B. Rehfeld. *API Discovery Core (draft-rehfeld-apix-core)*. IETF. <https://datatracker.ietf.org/doc/draft-rehfeld-apix-core/>
- [6] *TOON - Token-Oriented Object Notation*. <https://toonformat.dev>
- [7] *SEP-1576: Mitigating Token Bloat in MCP*. modelcontextprotocol, issue 1576. <https://github.com/modelcontextprotocol/modelcontextprotocol/issues/1576>
- [8] *Response size limit for MCP responses to prevent context overflow in AI agents*. modelcontextprotocol, discussion 2211. <https://github.com/modelcontextprotocol/modelcontextprotocol/discussions/2211>
- [9] *Headroom: compress tool outputs, logs, files, and RAG chunks before they reach the LLM*. <https://github.com/headroomlabs-ai/headroom>
- [10] Anthropic. *Code execution with MCP: building more efficient agents*. <https://www.anthropic.com/engineering/code-execution-with-mcp>

- [11] Anthropic. *Introducing advanced tool use on the Claude Developer Platform*. <https://www.anthropic.com/engineering/advanced-tool-use>
- [12] *Ethereum Signature Database (4byte directory)*. <https://www.4byte.directory>

TORPC spec and conformance vectors (CC0-1.0): <https://github.com/w3tech/torpc> · reference code and benchmark (Apache-2.0): <https://github.com/w3tech/torpc-js>